

软件工程基础

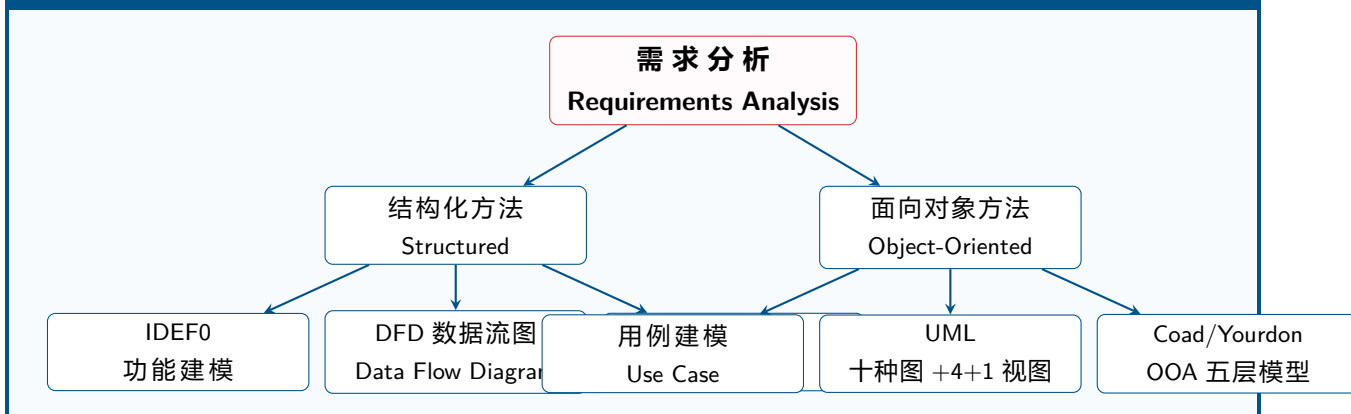
第 5 章—软件需求分析

Software Engineering Fundamentals —Chapter 5: Software Requirements Analysis

中南大学计算机学院 任胜兵

2026 年 6 月 1 日

本章知识图谱



- 掌握需求分析的基本概念、特点与需求收集方式 [p.2-6]
- 精通 DFD 数据流图的分层画法、数据词典与加工说明 [p.7-20]
- 掌握 IDEF0 功能建模（盒子/箭头/ICOM 码/结点树） [p.21-34]
- 掌握 IDEF1X 数据建模（实体/联系/属性/建模步骤） [p.35-42]
- 精通 UML 十种图的分类、画法与 4+1 视图架构 [p.43-65]
- 精通用例图及其建模步骤 [p.66-75]
- 掌握 Coad/Yourdon OOA 五层模型 [p.76-95]
- 了解需求规格说明 (SRS) 与需求验证 [p.96-99]

1 需求分析概述 (Overview of Requirements Analysis)

1.1 需求分析是什么? (What is Requirements Analysis?)

[p.2]

需求分析是指开发人员通过细致的调查分析, 详细、准确和完整地理解用户需要什么样的软件, 将用户**非形式的需求陈述**转化为**完整的需求定义**, 再将需求定义转换到相应的**需求规格说明**的过程。

通常, 把一整套的需求分析方法、技术和工具等的集合称为**建模方法** (Modeling Method)。

1.2 需求分析的特点 (Characteristics)

[p.3]

- **交流障碍** (Communication Barriers): 用户与开发人员之间存在理解偏差
- **需求易变性** (Requirements Volatility): 用户需求随时可能变化
- **不一致性** (Inconsistency): 不同用户的不同要求可能相互矛盾
- **不完整性** (Incompleteness): 用户难以一次性完整表达所有需求

解决策略 Solution

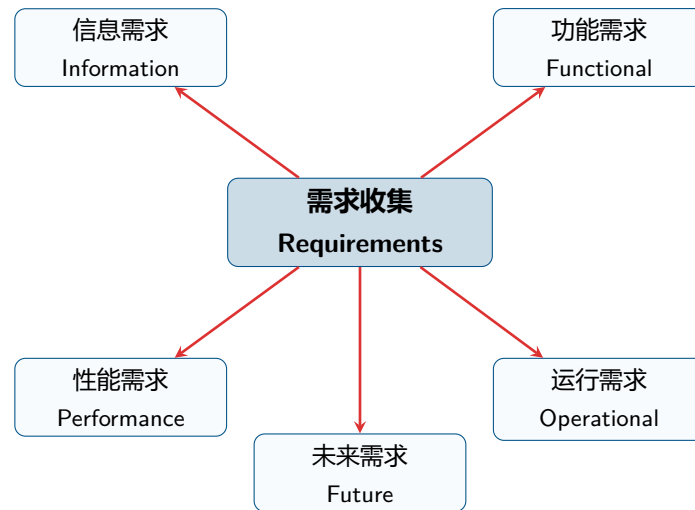
- 开发人员必须与有关人员密切配合, 充分交换意见
- 借助各种建模方法进行分析
- 对用户提出的各项要求应认真分析, 不能机械地全盘接受

1.3 需求收集 (Requirements Elicitation)

[p.4-6]

1.3.1 需求收集的内容 (Content)

[p.5]



1.3.2 需求收集的方式 (Methods)

[p.6]

- 访谈 (Interview)：程式化访谈 (Structured) 和非程式化访谈 (Unstructured)
- 问卷调查 (Questionnaire Survey)
- 场景使用 (Scenario-based)：模拟用户实际使用场景
- 用户资料收集 (User Documentation Collection)

2 数据流建模 (Data Flow Modeling (DFD))

[p.7-20]

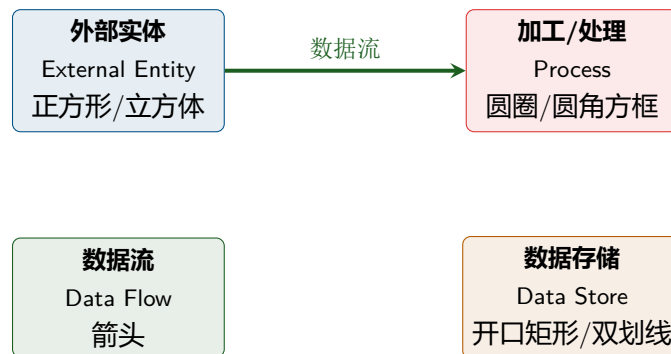
数据流建模方法是一种结构化分析方法，核心思想是自顶向下、逐层分解地定义系统需求。其特点是利用数据流图 (Data Flow Diagram, DFD) 来对用户需求进行分析，可用于分析任何应用系统的需求。

2.1 数据流图 (DFD —Data Flow Diagram)

[p.8]

2.1.1 四种基本符号 (Four Basic Symbols)

[p.8]

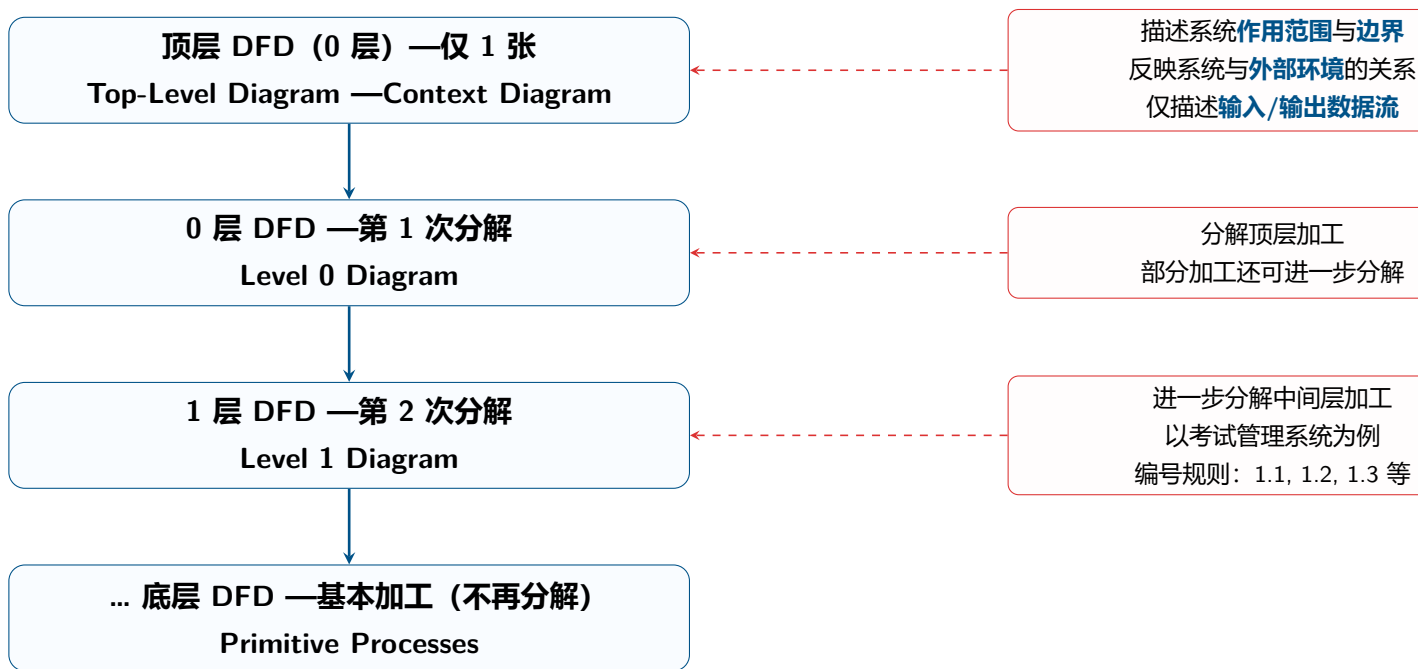


- **外部实体** (External Entity / Source/Sink)：数据的源点或终点，用正方形或立方体表示 [p.8]
- **加工** (Process)：对数据进行的变换处理，用圆圈或圆角方框表示 [p.8]
- **数据流** (Data Flow)：数据在系统中的流动方向，用箭头表示 [p.8]
- **数据存储** (Data Store)：数据的静态存储，用开口矩形框或双划线表示 [p.8]
- 扩展符号：* (与)、+ (或)、 $\oplus$ (异或) [p.8]

2.1.2 分层数据流图 (Hierarchical DFD)

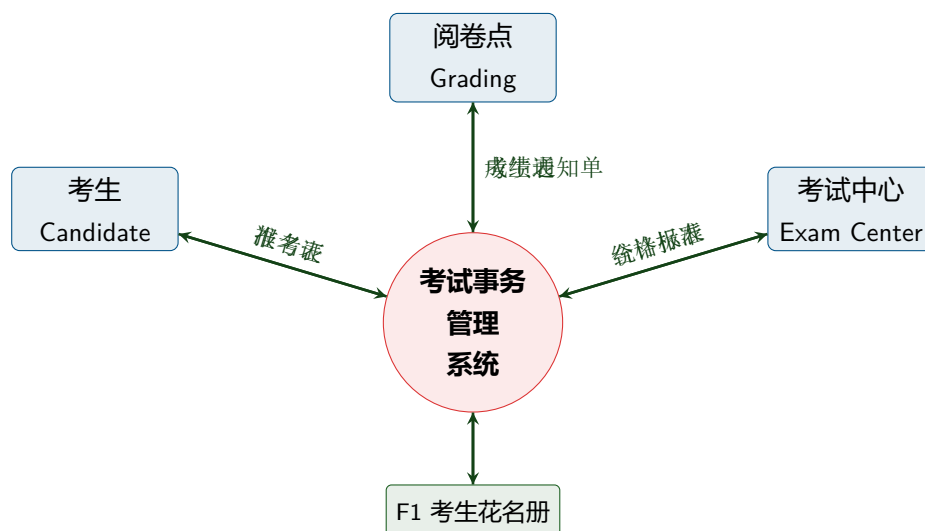
[p.9-12]

- 只用一张数据流图描述大型系统：难于一气呵成，且难于理解 [p.9]
- 分层 DFD 可以避免一次引入过多细节，有利于控制问题的复杂度 [p.9]
- 不同用户可只选择与本身有关或感兴趣的部分，不必阅读全图 [p.9]



[p.10-12]

1. 顶层数据流图 (Top-Level DFD)：描述整个系统的**作用范围**，说明系统**边界**，反映系统和外部环境之间的关系，即系统的**输入和输出数据流**。顶层数据流图**只有一张**。[p.10]
2. 中间层数据流图 (Intermediate DFD)：通过分解高层加工得到，其中有些加工还需进一步分解。[p.12]
3. 底层数据流图 (Primitive DFD)：由一些不必再进行分解的**基本加工** (Primitive Process) 组成。[p.11]



图：考试事务管理系统顶层 DFD (示例) [p.10]

2.2 数据词典 (Data Dictionary, DD)

[p.13-14]

数据词典 (Data Dictionary, DD) 是关于数据信息的集合, 是对数据流图中的**每个数据** (包括数据流和数据存储) 进行严格定义的场所, 以保持数据在系统中**的一致性**。[p.13]

- 组成描述符用于精确定义数据组成
- 数据词典卡片包含: **名称**、**别名**、**描述**、**组成**、**注释**等字段
- 可描述: 数据流、数据项、数据存储、扩展数据词典 [p.14]

2.3 加工说明 (Process Specification)

[p.15-19]

[p.15] 数据流图中的**基本加工**由于没有进一步分解得到子图, 因而需要加工说明来对其进行描述。描述的是**加工规则** (What) 而非实现细节 (How)。

1. **IPO 图** (Input/Process/Output Chart): 输入框、处理框、输出框三部分。常与层次图 (HC) 结合形成**HIPO 图**。[p.16]
2. **结构化语言 / PDL** (Program Design Language / Pseudo Code): 介于自然语言和形式语言之间的半形式语言, 使用 IF-THEN-ELSE、循环等控制结构。[p.17]
3. **判定表** (Decision Table): 适合条件组合引发不同动作的场景。[p.18]
4. **判定树** (Decision Tree): 判定表的图形化表示, 有时比判定表更直观。[p.19]

2.4 数据流建模步骤 (DFD Modeling Steps)

[p.20]

1. 画**顶层数据流图**——确定系统边界
2. 画**分层数据流图**——自顶向下逐层分解
3. 用**数据词典**定义数据流图中的所有数据
4. 用**加工说明**描述数据流图中的基本加工

3 IDEF0 功能建模 (IDEF0 Functional Modeling)

[p.21-34]

1981 年美国空军 ICAM 工程中开发了**IDEF** (ICAM DEFinition / Integration DEFinition) 方法族:

- **IDEF0**: 功能建模 (Functional Modeling)
- **IDEF1X**: 数据建模 (Data Modeling)
- **IDEF4**: 面向对象设计 (Object-Oriented Design)

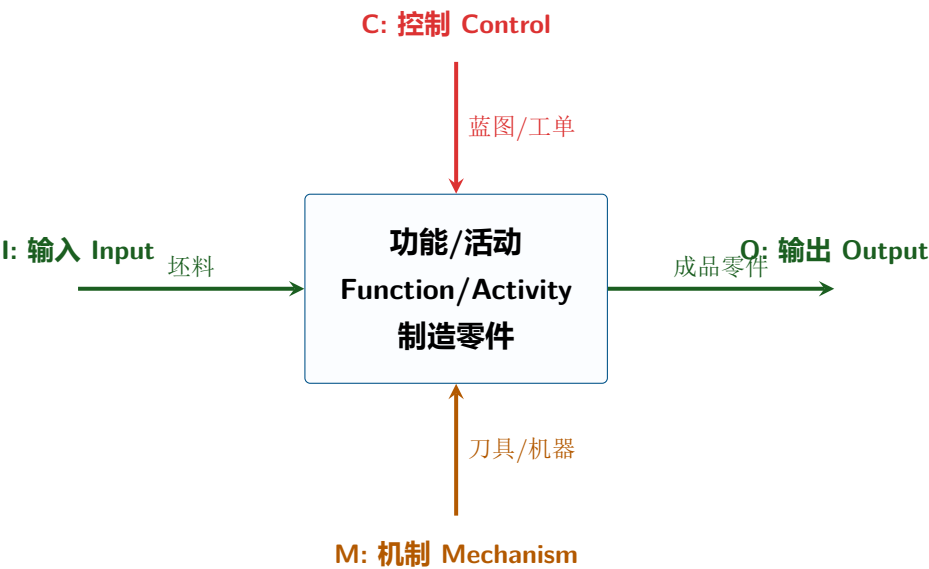
IDEF0 方法用严格的**自顶向下、逐层分解**的方式来构造系统的功能模型, 用 IDEF0 图来描述。[p.21]

3.1 IDEF0 图的基本元素 (Basic Elements)

[p.22-29]

3.1.1 盒子 Box (Function/Activity)

[p.22-23]



Input (输入) —Control (控制)—Output (输出)—Mechanism (机制)

盒子代表系统的**功能(活动)**; 箭头表示系统处理的**数据约束**, 可以是具体事物或抽象信息。
[p.22-23]

数量规则: 一张 IDEF0 图中的盒子**最多 6 个**, 子图形还要求**不少于 3 个**。[p.23]

3.1.2 箭头的种类 (Arrow Types)

[p.24–29]

箭头类型 Arrow Type	说明 Description
分支箭头 (Branching Arrow) [p.24]	表示多个活动需要同一种数据或同一种数据的不同成份。箭头代表数据约束而非数据流或执行顺序。在同一图上，若几个盒子所需的数据约束都满足，则这些活动可同时执行。
汇合箭头 (Merging Arrow) [p.25]	表示多个活动产生（或合成）同一种数据。
通道箭头 (Tunnel Arrow) [p.26]	表示箭头描述的数据约束不出现在子图（或父图）中。可使子图描述简化，避免在高层父图中出现信息过细的现象。
双向箭头 (Two-way Arrow) [p.27]	表示两个盒子描述的活动互为输入或互为控制，且将先触发的盒子画在较高位置上。
虚线箭头 (Dashed Arrow) [p.28]	表示活动触发的先后顺序（时序关系）。
选择箭头 (Selection Arrow) [p.29]	表示数据的选择关系（a 或 b）。

3.2 分层 IDEF0 图与 ICOM 码 (Hierarchical IDEF0 and ICOM Code)

[p.30–32]

[p.30] 为了明确分解过程中数据和图形间的关系，采用ICOM 码和结点号来进行标识：

- I (Input)：来自父盒子的输入边线
- C (Control)：来自父盒子的控制边线
- O (Output)：来自父盒子的输出边线
- M (Mechanism)：来自父盒子的机制边线

字母之后跟上数据约束在父盒子中的数字顺序号（编号顺序：从上至下，从左至右），以表明数据约束在父盒子中的位置。

(Node Tree)

[p.32] 结点号用来表明图形或盒子在分层 IDEF0 图中的位置。例如：

- A-0：系统内外关系图
- A0：顶层图
- A1, A2, ..., An：A0 的子图
- A21, A22, ..., A2m：A2 的子图

- 文字说明编号：图形结点号 + T（如 A12T）
- 参考图形编号：图形结点号 + F + 顺序号（如 A1F4）

3.3 IDEF0 建模步骤（IDEF0 Modeling Steps）

[p.34]

1. 确定建模的范围、观点及目的
2. 建立系统的内外关系图——即 **A-0 图**（Context Diagram）
3. 建立 A0 图的一系列子图——逐层分解
4. 书写文字说明

4 IDEF1X 数据建模 (IDEF1X Data Modeling)

[p.35-42]

- IDEF1X 方法是 IDEF1 方法的扩展版本：
- IDEF1 用来表示系统的**信息结构和语义**
 - IDEF1X 增强了图形的**表达能力**，丰富了语义和简化了开发过程 [p.35]

4.1 实体 (Entity)

[p.36]

实体 (Entity) 是具有相同属性或特征的现实或抽象事物的集合，集合中的一个元素称为实体的一个**实例 (Instance)**。在一张 IDEF1X 图中，一个实体只能出现**一次**。

- **独立实体 (Independent Entity)**：不依赖于其他实体而存在，用**直角矩形**表示 [p.36]
- **从属实体 (Dependent Entity)**：依赖于其他实体而存在，用**圆角矩形**表示 [p.36]
- 表示格式：实体名/实体编号（如：供应商/2） [p.36]

4.2 联系 (Relationship)

[p.37-40]

联系类型 Relationship	说明 Description
连接联系 (Connection Relationship) [p.38]	实体之间的基本联系，包括 可标定联系 (Identifying) （子实体依赖于父实体）和 非标定联系 (Non-identifying) （子实体不依赖父实体）。确定联系分为：一对一、一对多。
分类联系 (Categorization Relationship) [p.39]	描述实体之间的继承层次： • 完全分类联系 (Complete Categorization)：所有子类覆盖全部父类实例 • 不完全分类联系 (Incomplete Categorization)：子类不覆盖全部父类实例
非确定联系 (Non-specific Relationship) [p.40]	即 多对多 (M:N) 关系。需要转化为确定联系——通过引入 中间实体 (关联实体) 来转换。

4.3 属性 (Attribute)

[p.41]

- 属性表示现实或抽象事物的一种特性或性质
- 关键字属性 (Primary Key) : 唯一标识实体的属性
- 次关键字属性 (Alternate Key, AK) : 其他可标识属性
- 属性表示格式: 属性名位于实体框内, 关键字属性在上方

4.4 IDEF1X 建模步骤 (Modeling Steps)

[p.42]

目标和计划; [step] (s1) at (3.5,0) 阶段 1

定义实体; [step] (s2) at (7,0) 阶段 2

定义联系; [step] (s3) at (10.5,0) 阶段 3

定义键; [step] (s4) at (14,0) 阶段 4

定义属性;

[arr] (s0) – (s1); [arr] (s1) – (s2); [arr] (s2) – (s3); [arr] (s3) – (s4);

5 UML 建模语言 (Unified Modeling Language)

[p.43–65]

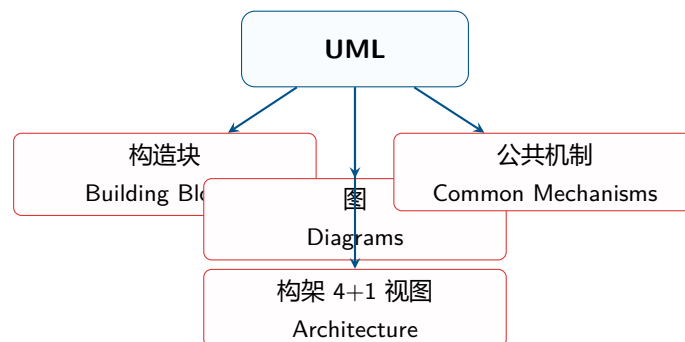
UML (Unified Modeling Language) 是一种可以应用于任何面向对象软件开发方法的标记法和语义语言。UML 融合了：

- **Booch** (G. Booch) 的 Booch 方法
- **OMT** (J. Rumbaugh) 的对象建模技术
- **OOSE** (I. Jacobson) 的面向对象软件工程

正如 G. Booch 所说：“如果你有好的思想，那它也是我们的。” [p.44–45, 63]

5.1 UML 的组成结构 (UML Structure)

[p.46]



- **构造块**：物件（结构/行为/分组/注释）+ 关系（关联/依赖/泛化/实现） [p.46]
- **图**：模型的图形表示（10 种图） [p.46]
- **公共机制**：规格说明、修饰、公共划分（类元与实例、接口与实现）、扩展机制（约束、构造型、标记值） [p.46, 49]
- **构架**：4+1 视图 [p.46, 50]

(Extension Mechanisms)

[p.49]

- **约束** (Constraint)：用花括号 {} 括起的文本字符串，说明必须维持为真的条件和规则
- **构造型** (Stereotype)：已有模型元素的变体，名称用书名号 《》 括起
- **标记值** (Tagged Value)：用花括号括起的标记值列表，如 {tag1=value1, tag2=value2}

5.2 UML 十种图总览 (Overview of 10 UML Diagrams)

[p.48, 51–62]

5.2.1 分类框架 Classification Framework

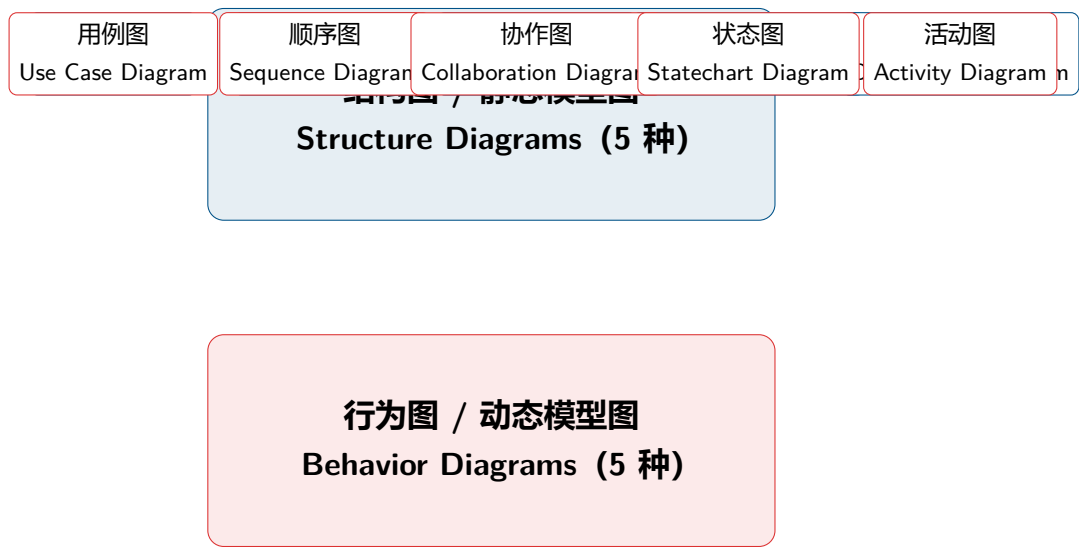
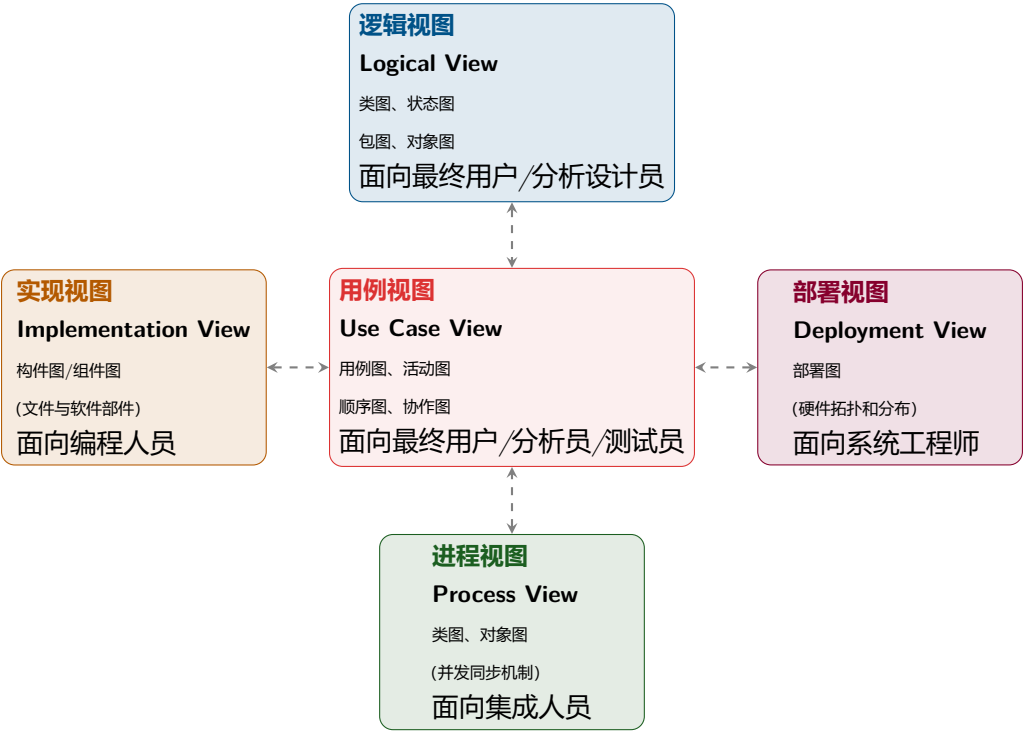


图 名 gram	Dia-	英文名	核心描述 Description
类图 [p.51-52]		Class Diagram	描述系统的 静态结构 ：类、接口及其关系（关联、泛化、依赖、实现、使用）。关键元素是 类元 （Classifier）。表示格式：名称栏、属性栏、操作栏。
对象图 [p.56]		Object Diagram	描述一组 对象 及其关系。对象是类的 具体实例 ，用矩阵形表示，标明对象名和属性取值。显示系统在某一时刻的 快照 。
包图 [p.53]		Package Diagram	由包或类组成，表示包与包、包与类之间的关系，用于描述系统的 分层结构 。包用 文件夹图形 来描述。
构件图 [p.54]		Component Diagram	描述软件各个组件（源代码、二进制、脚本、可执行文件等）之间的 依赖关系 。组件用 左侧带两个小矩形的大矩形 表示。
部署图 [p.55]		Deployment Diagram	显示运行时处理节点以及其中组件的配置，反映系统 硬件的物理拓扑结构 。节点用 三维矩形 表示。
用例图 [p.57]		Use Case Diagram	从系统 外部执行者 角度描述系统需提供哪些功能，指明功能的参与者。详见 §5.7。

图 名	Dia-gram	英文名	核心描述	Description
顺序图	[p.58]	Sequence diagram	Dia-	用来建模以 时间顺序 安排的对象间的交互。包含：对象生命线、激活、 同步/异步消息 、控制权。
协作图	[p.59]	Collaboration Diagram	Di-	建模对象或角色之间的交互，描述如何彼此通信。强调 空间结构 上的交互。与顺序图语义等价（可相互转换）。
状态图	[p.60]	Statechart diagram	Dia-	描述一个类对象所经历的各种 状态 以及事件发生时状态的 转移条件 。包含：状态、事件、转移、动作。
活动图	[p.61–62]	Activity Diagram		由状态图变化而来，描述需要执行的活动以及执行这些活动的 顺序 。包含：活动、分支、分叉汇合、 泳道 （Swimlane）（划分职责）。

5.3 4+1 视图架构 (4+1 View Architecture)

[p.50]



视图 View	英文名	描述与关注的图
用例视图	Use Case View	核心视图（+1）。描述系统行为、用户和系统的交互。包含：用例图、活动图、顺序图、协作图。面向最终用户、分析员和测试人员。
逻辑视图	Logical View	系统的面向对象模型，描述系统的静态结构和动态行为。包含：类图、状态图、包图、对象图。面向最终用户、分析设计人员。
进程视图	Process View	描述系统的并发和同步机制，包括进程、线程的组织。包含：类图、对象图（关注并发）。面向集成人员。
实现视图	Implementation View	描述用来发布实际系统的文件和软件部件，关注配置管理和系统组装。包含：构件图/组件图。面向编程人员。
部署视图	Deployment View	描述硬件拓扑结构和分布。包含：部署图。面向系统工程师。

5.4 UML 的特点（Features of UML）

[p.63]

- 统一了面向对象方法的基本概念：融合 Booch、OMT、OOSE
- 具有更强的建模能力
- 独立于特定的开发语言和开发过程

5.5 UML 的应用（Application of UML）

[p.64]

阶段 Phase	UML 图的选择 Application
需求分析	用例图 → 功能需求；类图 → 静态结构；状态图、顺序图、协作图 → 类之间所需的协作，实现用例。
设计	定义软件系统中的技术细节用到的类，如处理用户交互的类、处理数据的类、处理通信和并行性的类等。
实现	组件图 → 代码组件的物理结构及组件间关系；部署图 → 硬件拓扑结构和组件的分布。
测试	类图 → 单元测试；组件图、协作图 → 集成测试；用例图 → 确认测试。

RUP 统一过程 (Rational Unified Process)

针对 UML 的应用，Rational Software 公司提出了**统一过程** (Rational Unified Process, RUP) 方法，将核心工作流与 UML 各图有机结合。[p.65]

6 用例建模 (Use Case Modeling)

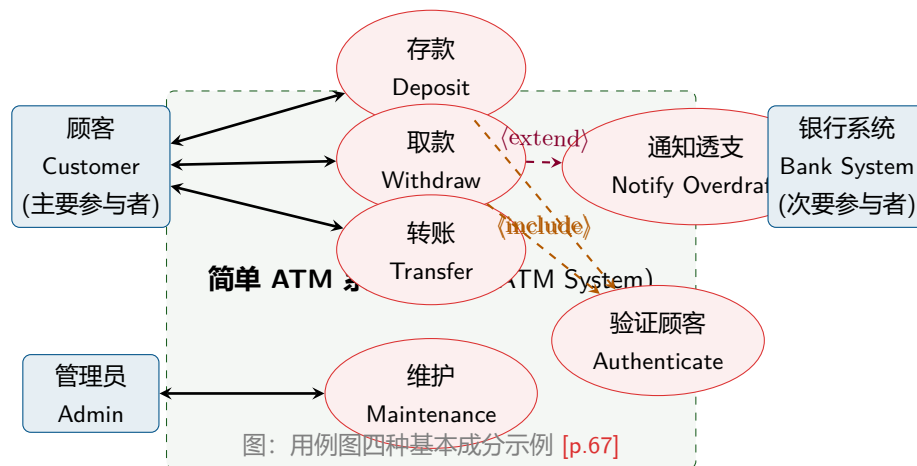
[p.66-75]

[p.66] 用例建模是一种从用户使用系统的角度来建立系统功能需求模型的方法。它既非从数据模型开始，亦非从系统数据流着手，而是从组成系统的实际操作入手。

用例图是用例建模的主要成果，从系统外部执行者的角度来描述系统需要提供哪些功能以及谁使用这些功能。

6.1 用例图的基本成份 (Basic Elements of Use Case Diagram)

[p.67]



6.1.1 四种基本成份 (Four Components)

[p.67]

1. 系统 (System)：用方框表示系统边界
2. 参与者 (Actor)：系统外部与系统交互的用户或其他系统
 - 主要参与者 (Primary Actor)：主动发起用例
 - 次要参与者 (Secondary Actor)：被动参与用例
3. 用例 (Use Case)：用椭圆表示，系统提供的功能单元
4. 关系 (Relationship)：
 - 通信关系 (Communication)：参与者与用例之间的连线
 - 包含关系 (Include) <<include>>：一个用例必然包含另一个用例的行为（必须执行）
 - 扩展关系 (Extend) <<extend>>：一个用例在特定条件下扩展另一个用例的行为（可选执行）
 - 泛化关系 (Generalization)：参与者之间或用例之间的继承关系

6.2 用例描述方法 (Use Case Description Methods)

[p.68-73]

方法 Method	特点与说明 Description
简单文字 [p.69]	自然语言描述，简明直接。如：”用例’获取呼叫历史’使客户可以查阅账上已付费的所有呼叫的细节。”
模板 [p.70]	使用结构化模板，包含用例名称、参与者、前置条件、后置条件、基本事件流、备选事件流等字段。最常用。
表格 [p.71]	用表格形式组织用例信息，清晰条理。
图形 [p.72]	使用 UML 活动图或顺序图等图形化描述用例流程。
形式化语言（Z 语言） [p.73]	使用严格的数学语言（如 Z 语言）进行精确描述。形式化程度最高。

6.3 用例建模步骤 (Use Case Modeling Steps)

[p.75]

- 1. 找出系统的参与者和用例
 - 参与者标准：至少能确定一个用户来扮演；不同参与者实例间角色重叠最少
 - 用例标准：有价值的结果和特定参与者
- 2. 区分用例的优先次序
- 3. 详细描述每个用例
- 4. 构造用户界面原型
- 5. 构造用例图
 - 用例的结构关系应尽量反映真实情况
 - 将每个单独的用例视为一个单独的制品
 - 避免从功能上分解用例

7 对象建模—Coad/Yourdon OOA

[p.76-95]

[p.76] 对象建模的主要任务是了解某个特定应用问题域内所涉及的对象，以及各种各样的结构和通信关系。本节介绍Coad/Yourdon面向对象分析方法来进行对象建模，在表示符号上采用 UML 建模语言提供的标记法。

Coad/Yourdon OOA 方法采用五层模型：主题层、类与对象层、结构层、属性层、服务层。

7.1 确定对象与类（类与对象层）（Identifying Objects & Classes）

[p.77-80]

- 对象（Object）：现实世界某些事物的一个抽象，反映该事物在系统中需要保存的信息和发挥作用
- 类（Class）：具有相同属性和操作的一组对象的集合

UML 中类的表示：矩形框分三栏——名称栏、属性栏、操作栏。对象表示为矩形框，标注对象名和属性取值。[p.77]

7.1.1 确定对象/类的方法

[p.78]

1. 三视图模型法：实体-关系模型 + 数据流模型 + 状态-迁移模型
2. 基于语言的信息分析法：短语频率分析、矩阵分析
3. CRC 卡片（Class Responsibility Collaboration）：类名 + 职责 + 协作类

电梯控制系统示例（Elevator Control System Example）

[p.79-80]

- 系统：50 层建筑中 5 部电梯的调度与控制
- 标识出的对象/类：电梯、召唤请求面板、到达面板、目的地面板、楼层、超载传感器、到达事件、目的地事件、召唤事件
- 注意：对象/类不可能被证明绝对正确或错误；用户偏好对模型最后选择有很大影响

7.2 标识结构（结构层）（Identifying Structures）

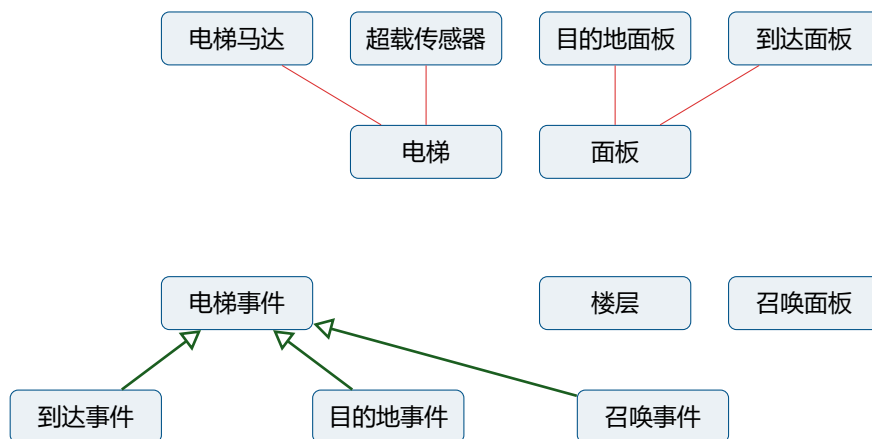
[p.81-83]

[p.81] 现实世界的事物之间存在两种主要关系：

1. 继承关系 (Inheritance / Generalization-Specialization)：一般类与特殊类之间的关系 (is-a)
2. 整体/部分关系 (Whole-Part)：
 - 聚合关系 (Aggregation)：弱的整体-部分关系（部分可独立存在）
 - 组合关系 (Composition)：强的整体-部分关系（部分不能独立存在，同生共灭）

[p.82]

- 继承关系：自顶向下（从一般到特殊）或自底向上（从特殊到一般）方式
- 整体/部分关系：
 - 把每个对象看成一个整体，考虑：什么是它的部分或组成成份？
 - 把每个对象看成一个部件，考虑：该对象适合什么样的组装整体？



图：电梯控制系统结构层示例 [p.83]

7.3 标识主题（主题层） (Identifying Subjects)

[p.84–86]

[p.84] 在对象建模过程中，大多数模型都是非层次式的。Coad/Yourdon 方法使用主题来处理规模比较大的复杂模型：

- 使用主题可以控制一个用户必须同时考虑的模式数目
- 使用主题有助于了解模型的总体概貌
- 在 UML 中，包 (Package) 可用来表示主题

[p.85]

1. 对每一个**结构**增加一个相应的主题
2. 对每个**对象**增加一个相应的主题
3. 将紧密耦合的主题**组合**起来得到更好的主题
4. 如果主题数目不符合**7±2 规则**，则应进一步精炼主题

选择主题时可依据：子问题域、子系统、组织或地域来区分。注意：不应用功能分解方法。

电梯控制系统主题层示例

[p.86]

- ”电梯管理”主题：电梯、电梯马达、超载传感器、到达面板、目的地面板、面板——主要用于控制硬件
- ”电梯调度”主题：电梯事件、到达事件、召唤事件、目的地事件、楼层、召唤面板——主要检测事件发生和对电梯进行调度

由于电梯控制系统规模不是很大，也可不必引入主题层。

7.4 定义属性及实例关联（属性层）（Defining Attributes & Instance Connections）

[p.87-91]

[p.87-89]

- **属性**（Attribute）：对象/类的数据单元，定义属性使得问题域更加明确
- 在分析阶段，一般不用属性来表示对象/类之间的关系，而使用**实例关联**来描述
- **实例关联**（Instance Connection）：可看成是一种事务规则或应用论域限制

属性的三种类型：

1. **描述性属性**（Descriptive）：如”身高”、”颜色”
2. **命名性属性**（Naming）：如”姓名”、”学生”
3. **参考属性**（Referential）：如”系别”（引用其他对象）

属性的确定既与问题域有关，也与目标系统的任务有关；定义属性应该在**原子概念的层次上**进行。对于分类结构，应利用**继承机制**来定位属性的位置。[p.89]

7.4.1 UML 中实例关联的表示

[p.88]

- **关联名称**（Association Name）：描述关联含义
- **角色名称**（Role Name）：描述类在关联中的角色
- **多重性**（Multiplicity）：形式为 minimum..maximum（如 1..*, 0..1）
- **导航性**（Navigability）：箭头表示消息传递方向；无导航标识表示双向

7.5 定义服务及消息连接（服务层）（Defining Services & Message Connections）

[p.92-95]

[p.92-93]

- **服务**（Service）：即对象/类的**操作**（Operation），反映对象的动态行为
- **消息**（Message）：对象/类之间在行为上的**唯一联系方式**
- 在 UML 类图中，可采用实例关联的**导航性**来表示消息连接；也可采用协作图或顺序图来描述

[p.92]

- 考察系统的**责任**
- 研究**问题域**
- 分析对象/类的**状态**或追踪对象/类的**执行线路**
- 用例模型中的**事件流**

[p.93]

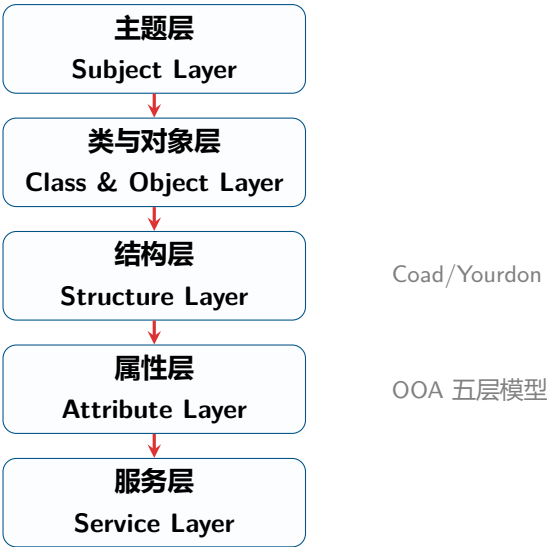
- 在已存在实例连接的对象/类和分类结构之间增加消息连接
- 检查对象/类和分类结构，寻找一个实例所需要的另一个实例的操作

电梯控制系统——服务层示例

[p.94-95] 基于事件-响应模式，针对 7 种事件定义服务：

1. 召唤电梯、目的地请求、电梯到达调度楼层、电梯到达非调度楼层、电梯就绪、电梯超载、电梯未超载

例如**召唤电梯事件**：识别召唤请求 → 报告召唤事件 → 更新召唤面板



8 需求规格说明与验证 (SRS & Verification)

[p.96–99]

8.1 需求规格说明 (Software Requirements Specification, SRS)

[p.96–97]

- 为用户、分析人员、设计人员和测试人员之间的理解和交流提供方便 [p.96]
- 是系统设计、测试和验收的依据 [p.96]
- 起着控制系统演化过程的作用，追加需求应结合 SRS 来考虑 [p.96]

8.1.1 需求规格说明方法

[p.97]

方法类型 Type	说明 Description
形式化方法 (Formal Methods)	采用完全精确的语法和语义来表达。分为五类： – 基于模型的方法（如Z 语言） – 代数方法（如OBJ 语言） – 过程代数方法（如CSP通信顺序进程） – 基于逻辑的方法（如时态逻辑） – 基于网络的方法（如Petri 网）
非形式化方法 (Informal Methods)	采用自然语言描述，可用图表及其它符号帮助理解，也可用标准化的格式来编制。

8.2 需求验证 (Requirements Verification)

[p.98]

大量统计数字表明，软件系统中15%的错误起源于需求的错误。 [p.98]

(Characteristics of Effective SRS)

[p.98]

- 正确性 (Correctness)
- 无歧义性 (Unambiguousness)
- 完整性 (Completeness)
- 一致性 (Consistency)
- 可验证性 (Verifiability)
- 可理解性 (Understandability)
- 可修改性 (Modifiability)
- 可追踪性 (Traceability)
- 注释 (Annotation)

一旦需求验证完成，需求规格说明就变成了软件开发的合同。尽管允许事后修改需求规格说明，但用户应该注意的是：每种事后修改都是对软件范围的扩展，因此可能增加成本和/或延长软件项目进度。[p.98]

9 本章小结 (Chapter Summary)

[p.99]

1. 需求分析的结果定义了待开发系统的范围，即必须”做什么”，是后续开发活动的基础 [p.99]

2. 需求分析存在许多困难或特点（交流障碍、需求易变性、不一致性、不完整性），需要用户和开发人员密切配合，充分交流，借助各种建模方法 [p.3, 99]

3. 结构化方法的主要特点之一是自顶向下、逐层细化：DFD 分层数据流图 → 数据词典 → 加工说明；IDEF0 功能建模；IDEF1X 数据建模 [p.99]

4. 面向对象方法采用更接近人类思维方式的形式来刻画实际系统：对象、类、继承和消息通信。核心工具是 UML（十种图 +4+1 视图）、用例建模和 Coad/Yourdon OOA 五层模型 [p.99]

两大方法论对比 Comparison of Two Paradigms

维度	结构化方法	面向对象方法
核 心 思 想	自顶向下、逐层分解	对象、类、继承、消息通信
分 析 工 具	DFD 数据流图、IDEF0、IDEF1X	UML（十种图）、用例图
建 模 视 角	功能 + 数据分离	功能 + 数据封装为对象
主 要 成 果	DFD 分层图 + 数据词典 + 加工说明	用例图 + 类图 + 顺序图 + OOA 模型
方 法 特 点	过程驱动，强调数据变换	数据驱动，接近人类思维方式
适 用 场 景	数据处理密集型系统	复杂交互式系统

各类图快速对照表 Quick Reference of All Diagrams			
图名	英文名	分类	一句话描述
DFD 数据流图	Data Flow Diagram	结构化	自顶向下描述数据流动与变换
IDEF0 图	IDEF0 Diagram	结构化	功能建模：ICOM 盒子与箭头
IDEF1X 图	IDEF1X Diagram	结构化	数据建模：实体/联系/属性
类图	Class Diagram	UML 静态	系统静态结构：类与关系
对象图	Object Diagram	UML 静态	系统快照：对象与关系
包图	Package Diagram	UML 静态	系统分层结构
构件图	Component Diagram	UML 静态	软件组件依赖关系
部署图	Deployment Diagram	UML 静态	硬件拓扑与组件分布
用例图	Use Case Diagram	UML 动态	系统功能与参与者
顺序图	Sequence Diagram	UML 动态	时间顺序的对象交互
协作图	Collaboration Diagram	UML 动态	空间结构的对象交互
状态图	Statechart Diagram	UML 动态	对象状态与转移条件
活动图	Activity Diagram	UML 动态	活动执行顺序与泳道